FX

Effect Interpreter와 안전한 코드 변경

모델의 tool proposal을 schema·권한·현재 상태·실제 효과의 결합 문제로 읽는다

> **도구 호출의 핵심은 함수를 실행하는 것이 아니라, 그 실행이 지금 이 대상에 허용되고 여전히 유효한지를 판정하는 일이다.**

대상 snapshot
v2.1.88 / 2ca5ddabfed5

설명 구조
조건 · 이론 · 구현 · 반례

제3자 복원 소스에 대한 교육용 분석이다. 코드 관찰, 이론적 비유, 개선 제안을 구분하며 누락 모듈을 추정해 채우지 않는다.

모델 출력은 신뢰할 수 없는 프로그램이다

tool_use를 단순 데이터로 보면 무엇을 놓치는가?

성립 조건

tool name과 arguments는 파일·shell·network 효과를 표현할 수 있다.

이론 렌즈

interpreter는 syntax를 읽고 semantics를 선택하며, protection system은 effect를 중재한다.

코드의 구체화

registry lookup→parse→validate→permission→call→result mapping이 작은 실행 pipeline을 이룬다.

깨지는 지점

JSON이 valid하다는 이유로 실행하면 model error와 adversarial content가 곧 authority가 된다.

해석 tool runtime은 model proposal language의 interpreter이자 policy enforcement boundary다.

근거와 연결 src/services/tools/toolExecution.ts · src/Tool.ts · MCP Tools

valid는 하나의 boolean이 아니다

> 도구 입력이 실행될 자격을 얻으려면 무엇이 모두 참이어야 하는가?

Representable ∧ Semantically admissible ∧ Authorized ∧ Current-state valid

판정	질문	대표 실패
표현	schema에 맞는가	필수 field 누락
의미	이 tool의 precondition을 만족하는가	읽지 않은 파일 편집
권한	이 subject가 이 effect를 해도 되는가	deny rule/path 범위
현재성	판정 뒤 resource가 그대로인가	mtime/content 변경

근거와 연결 src/services/tools/toolExecution.ts · JSON Schema

permission은 호출 전 한 번 묻는 UI가 아니다

> complete mediation은 어디에 나타나야 하는가?

성립 조건

각 resource access는 subject, action, target, current policy에 따라 달라진다.

이론 렌즈

Saltzer-Schroeder의 complete mediation과 least privilege는 모든 접근의 중재와 최소 권한을 요구한다.

코드의 구체화

general permission context와 input-specific tool validation이 호출 직전에 결합된다.

깨지는 지점

tool 등록 때 “safe”로 분류한 뒤 입력별 target을 다시 보지 않으면 권한이 너무 넓어진다.

해석	permission boundary는 static tool classification보다 실제 effect request 가까이에 있어야 한다.

근거와 연결 src/Tool.ts · src/services/tools/toolExecution.ts · NIST least privilege

하나의 Tool 타입에 서로 다른 변화 이유가 모여 있다

> 강한 계약과 과결합을 어떻게 구분하는가?

facet	consumer	변화 압력
model schema/prompt	provider/model	capability 표현
effect implementation	filesystem/process	runtime·platform
permission/validation	policy/user	security·authority
result mapping	model/protocol	pairing·error semantics
presentation	React/Ink	interactive UX
telemetry/search	operations	관측·discoverability

한 object가 end-to-end coherence를 제공하는 장점과, 독립 변화가 모두 같은 타입을 흔드는 비용이 동시에 존재한다. 분해는 facet 사이 invariant를 새 adapter가 보존할 때만 개선이다.

근거와 연결 src/Tool.ts

기본값은 보이지 않는 정책이다

false 하나가 어떤 safety claim을 만드는가?

성립 조건

많은 tool이 공통 builder로 생성되어 생략된 field를 default로 받는다.

이론 렌즈

secure default는 미지정 상태에서 권한을 넓히지 않아야 한다.

코드의 구체화

concurrency/read-only/destructive 관련 default가 registry 전체의 behavior floor를 만든다.

깨지는 지점

`isDestructive=false`를 "안전함"으로 해석하면 tool-specific partial effect를 놓친다.

해석 default는 편의 값이 아니라 새 tool이 증명하기 전까지 적용되는 보수적 policy여야 한다.

근거와 연결 src/Tool.ts

FileEdit의 읽기 요구는 UX 규칙이 아니다

왜 모델이 파일을 읽었다는 사실이 write precondition이 되는가?

성립 조건

편집 proposal은 읽은 content와 위치를 전제로 한다.

이론 렌즈

optimistic concurrency는 작업 중 lock 대신 commit 직전 관찰 version을 검증한다.

코드의 구체화

read state, full-read 요구, content/mtime 검사가 edit validation과 critical section을 구성한다.

깨지는 지점

read와 write 사이 외부 변경을 무시하면 syntactically valid patch가 다른 program을 손상한다.

해석 read-before-write는 model grounding과 concurrency control을 같은 evidence에 묶는다.

근거와 연결 src/tools/FileEditTool/FileEditTool.ts · Kung & Robinson OCC

검사 직후의 await가 correctness를 무너뜨릴 수 있다

왜 critical section 주석이 구조적 계약인가?

성립 조건

검사 대상은 다른 process나 user가 바꿀 수 있는 shared file이다.

이론 렌즈

TOCTOU는 check와 use 사이 property가 변해 protection assumption이 깨지는 문제다.

코드의 구체화

staleness check와 write 사이 await를 두지 말라는 경계가 코드에 명시된다.

깨지는 지점

permission prompt, telemetry flush, LSP query를 사이에 넣으면 race window가 다시 열린다.

해석 async code에서 “await가 없음”은 미세 최적화가 아니라 atomicity approximation의 일부다.

근거와 연결 src/tools/FileEditTool/FileEditTool.ts · CWE-367

파일 편집은 database transaction이 아니다

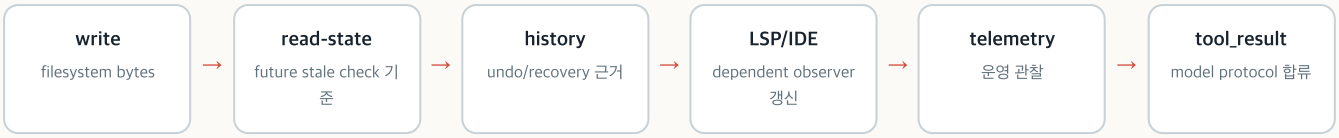
OCC 비유는 어디서 끝나는가?

database OCC	FileEdit에서 대응	남는 차이
read set/version	read state + mtime/content	filesystem identity·symlink
validation phase	write 직전 stale check	여러 파일 atomicity 없음
write phase	파일 content 갱신	crash durability/rollback 별도
abort/retry	stale 오류 후 재읽기	모델 재추론과 user intent 변화

경계 “낙관적 동시성 모양”은 유효하지만 serializable transaction이라고 부르면 과장이다.

write 이후에도 effect는 끝나지 않는다

> 파일 bytes 변경과 제품 상태 완료는 같은가?



이 순서는 effect 자체와 projection/observation을 구분하기 위해서만 제시한다. 하나가 실패했을 때 write를 되돌릴 수 있는지, degraded success로 볼지, 다음 turn에 어떤 사실을 알려야 하는지가 핵심이다.

근거와 연결 src/tools/FileEditTool/FileEditTool.ts

Bash는 문자열이 아니라 또 하나의 언어다

prefix allowlist가 왜 쉽게 깨지는가?

성립 조건

shell은 quoting, substitution, pipelines, redirection, env, wrappers를 해석한다.

이론 렌즈

injection은 data가 command structure를 바꿀 때 발생한다.

코드의 구체화

bashSecurity와 bashPermissions가 syntax와 command semantics를 여러 단계에서 해석한다.

깨지는 지점

첫 token만 안전해도 `&&`, `$()`, redirection, env PATH, wrapper가 다른 effect를 추가한다.

해석 shell permission은 command-name 분류가 아니라 program analysis의 축소판이다.

근거와 연결 src/tools/BashTool/bashSecurity.ts · src/tools/BashTool/bashPermissions.ts · OWASP command injection

완전한 shell 해석과 실용적 policy 사이에는 간극이 있다

어떤 방어를 겹쳐야 하는가?

층	역할	단독 한계
prompt	모델이 위험 행동을 덜 제안	adversarial/오류에 취약
parser/normalization	실제 구조 추출	shell 방언·dynamic expansion
allow/deny policy	알려진 capability 제한	semantic 우회·새 command
user confirmation	intent 확인	피로·복잡한 command 이해
sandbox	blast radius 제한	설정·platform gap
telemetry/recovery	사후 진단	이미 발생한 비가역 효과

defense in depth 어느 한 층도 “안전한 Bash”를 증명하지 않는다. 각 층은 다른 failure mode를 줄인다.

병렬성은 tool 이름이 아니라 입력별 효과에서 결정된다

> Read는 항상 병렬이고 Bash는 항상 직렬인가?

성립 조건

같은 tool도 target과 arguments에 따라 resource 충돌 가능성이 달라진다.

이론 렌즈

commutativity가 없는 effect의 순서 변경은 observable behavior를 바꾼다.

코드의 구체화

`isConcurrencySafe(input)`로 연속 safe group을 병렬화하고 unsafe group은 직렬화한다.

깨지는 지점

classifier parse 오류를 safe로 처리하거나 alias resolution 뒤 의미가 달라지면 race가 발생한다.

해석	concurrency policy는 type-level marker보다 effect summary에 가까워야 한다.

근거와 연결 src/services/tools/toolOrchestration.ts

결과 배열 순서와 실행 완료 순서는 다르다

병렬 실행 후 다음 model turn은 무엇을 보아야 하는가?

병렬 tool은 완료 시간이 달라도 original tool_use order에 맞추어 result와 context modifier가 합류해야 protocol과 reproducibility가 유지된다. 따라서 orchestration은 throughput뿐 아니라 stable observation order를 소유한다.

설계 질문	완료 순서를 노출할 소비자와 semantic 순서를 요구하는 소비자를 분리했는가?

근거와 연결 src/services/tools/toolOrchestration.ts · src/query.ts

확장 protocol은 authority를 상속하지 않는다

외부 MCP tool이 등록되면 실행 자격도 생기는가?

성립 조건

MCP server는 model-controlled tool surface를 제공할 수 있다.

이론 렌즈

capability discovery, schema validity, trust, effect admission은 별도 판단이다.

코드의 구체화

MCP client/adapter 결과가 내부 tool execution contract로 들어온다.

깨지는 지점

remote description을 local safety classification으로 그대로 받아들이면 provider가 authority owner가 된다.

해석	확장성은 더 많은 tool을 찾는 능력이고, 안전성은 각 tool의 effect를 다시 local policy에 투영하는 능력이다.

근거와 연결 src/services/mcp/client.ts · MCP server primitives

오류는 “아무 일도 없었다”를 뜻하지 않는다

> tool call이 throw하면 어떤 상태를 가정해야 하는가?

사건	가능한 외부 상태	필요한 observation
spawn 후 abort	process가 계속 실행/자식 생성	pid·signal·cleanup receipt
write 후 LSP 실패	file은 변경됨	filesystem reread + degraded result
network timeout	remote commit 여부 unknown	idempotency key/read-back
stream disconnect	tool proposal 일부 수신	실행 시작 여부와 request id

오류 타입은 control-flow 정보가 아니라 effect uncertainty를 표현해야 한다. unknown을 false로 바꾸면 retry가 중복 효과를 만든다.

tool_result는 관찰이자 다음 프로그램의 입력이다

오류 메시지의 구조가 왜 중요한가?

성립 조건

모델은 result를 읽고 다음 tool proposal을 만든다.

이론 렌즈

interpreter의 observation은 다음 transition을 결정하는 typed feedback이다.

코드의 구체화

result mapping은 model content, UI display, error status를 구분한다.

깨지는 지점

raw exception을 그대로 넣으면 비밀·noise·provider-incompatible shape가 유입되고, 너무 요약하면 recovery evidence가 사라진다.

해석	result contract는 effect truth를 최소 충분하게 전달하는 anti-corruption layer다.

근거와 연결 src/Tool.ts · src/services/tools/toolExecution.ts

보안은 위험 명사보다 공격 가능한 scenario로 읽는다

Bash permission 변경을 어떻게 검토할까?

source	stimulus	environment	artifact	response	measure
model 또는 pasted untrusted text	quote 속 substitution과 redirect 제안	user가 broad allow rule을 보유	parser + permission + sandbox	구조를 보수적으로 분해하거나 확인 요구	unauthorized effect 0, false block 별도 측정

허용률 평균으로 unauthorized effect를 보상하지 않는다. 안전 hard floor와 usability observation을 분리한다.

effect runtime은 negative path가 본체다

> 정상 호출 snapshot 외에 무엇을 시험해야 하는가?

case	oracle	보존 불변조건
schema 오류	call 미실행	syntax≠authority
permission deny	effect 0 + paired result	protocol closure
stale edit	write 0	read evidence freshness
abort during effect	partial-state receipt	unknown 보존
parallel conflict	deterministic serialization	resource consistency
adapter throw	sanitized observation	secret/protocol safety

안전한 tool은 작은 함수가 아니라 작은 제어면이다

이 권에서 남겨야 할 한 문장은 무엇인가?

> **모델이 무엇을 제안했는지보다, 누가 어떤 현재 상태에서 어떤 효과를 허용했고 무엇을 다시 관찰했는지가 더 중요하다.**

Tool abstraction의 성공은 call을 통일하는 데 있지 않다. 서로 다른 validation proposition과 partial-effect semantics를 숨기지 않으면서 동일한 turn protocol로 합류시키는 데 있다.